

Meeting Master - AI Scrum Assistant

Comprehensive Design Document

Version: 1.0

Created: January 29, 2026

Status: In Development

Table of Contents

- 1. Executive Summary
- 2. Problem Analysis
- 3. Competitor Analysis
- 4. Solution Architecture
- 5. Feature Specification
- 6. AI Prompt Engineering
- 7. Database Schema
- 8. API Design
- 9. Frontend Design
- 10. Technology Stack
- 11. Deployment Plan

Executive Summary

Meeting Master is an AI-powered meeting assistant designed for **ad-hoc workplace meetings** - the 80% of meetings that happen spontaneously without calendar invites. One tap to record, automatic transcription, intelligent task extraction, calendar booking, and MoM distribution.

Key Differentiators

Feature	Competitors	Meeting Master
Ad-hoc Recording	Requires pre-scheduled call	One-tap instant start
Language Support	English-only	Hindi, English, Hinglish
Task Intelligence	Basic extraction	Intelligent deadlines, context-aware
Calendar Booking	Manual/Integration	Auto-suggest with smart dates
Privacy	Cloud-dependent	BYOK (Bring Your Own Key) option
Cost	\$10-20/user/month	Free tier + \$5/mo premium

Target Users

- 1. **Primary:** Indian startups, IT companies (5-50 employees)
- 2. **Secondary:** Freelancers, consultants, remote teams

Problem Analysis

The Ad-Hoc Meeting Gap

Research Finding: 80% of workplace meetings are unscheduled (Gartner 2025)

```
Types of Meetings by Volume:
├─ Scheduled Meetings (20%)
│   └─ Well-served by Fireflies, Otter, Zoom AI
└─ Ad-Hoc Meetings (80%)
    ├─ Quick standups at desk
    ├─ Corridor discussions
    ├─ Impromptu brainstorm
    ├─ Client calls on mobile
    └─ No good solution exists
```

Pain Points from User Research

Pain Point	User Quote	Impact
Setup friction	"By the time I open Fireflies, meeting is over"	Lost meeting content
Calendar dependency	"Can't use without booking a Zoom call"	Excludes informal meetings
Language barriers	"My team speaks Hindi, tools only do English"	Poor transcription quality
Privacy concerns	"Don't want audio on third-party servers"	Trust issues
Task ambiguity	"Tasks extracted are vague, no deadlines"	Follow-up chaos
Cost per user	"\$10/user adds up for 20-person team"	Budget constraints

Market Opportunity

```
India IT Workforce: 5.4 million (NASSCOM 2025)
├─ Startups (< 50 employees): 2.1 million
├─ SMBs (50-500): 1.8 million
└─ Enterprise: 1.5 million

Target Addressable Market: 2.1M users × $5/mo = $126M ARR potential
```

Competitor Analysis

Detailed Comparison Matrix

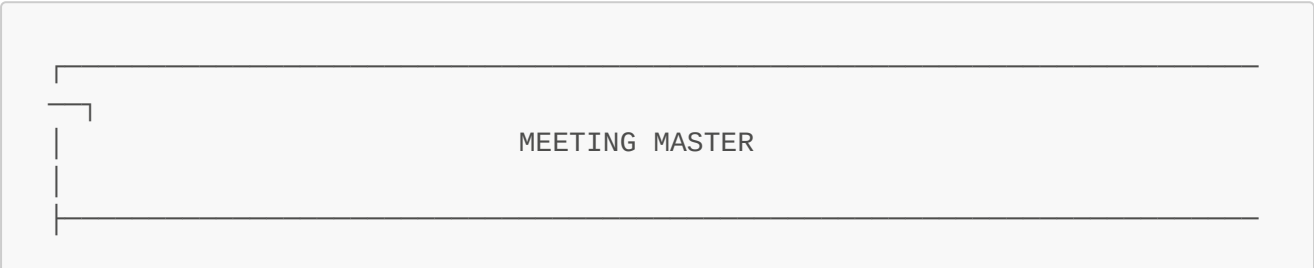
Feature	Fireflies.ai	Otter.ai	Gemini	Fathom	Manus	Meeting Master
Ad-hoc mobile	△ Partial	☐ Yes	△ Partial	☐ Yes	☐ Yes	☐ One-tap
Pre-schedule needed	☐ Yes	△ Optional	☐ Yes	☐ No	☐ No	☐ No
Speaker ID	☐ Yes	☐ Yes	☐ Yes	☐ Yes	△ Basic	☐ Yes
Hindi support	☐ No	☐ No	△ Basic	☐ No	☐ No	☐ Native
Hinglish support	☐ No	☐ No	☐ No	☐ No	☐ No	☐ Yes
Auto calendar	☐ Integrations	△ Partial	☐ Yes	☐ No	☐ No	☐ Smart
Task deadlines	△ Manual	△ Manual	△ Manual	△ Manual	☐ No	☐ AI-inferred
MoM email	☐ Yes	☐ Yes	☐ Yes	☐ Yes	☐ Yes	☐ Editable
BYOK option	☐ No	☐ No	☐ No	☐ No	☐ No	☐ Yes
WhatsApp share	☐ No	☐ No	☐ No	☐ No	☐ No	☐ Yes
Offline mode	☐ No	☐ No	☐ No	☐ No	☐ No	☐ Planned
Price	\$10+/user	\$10+/user	Workspace	Free tier	Free	\$5/user

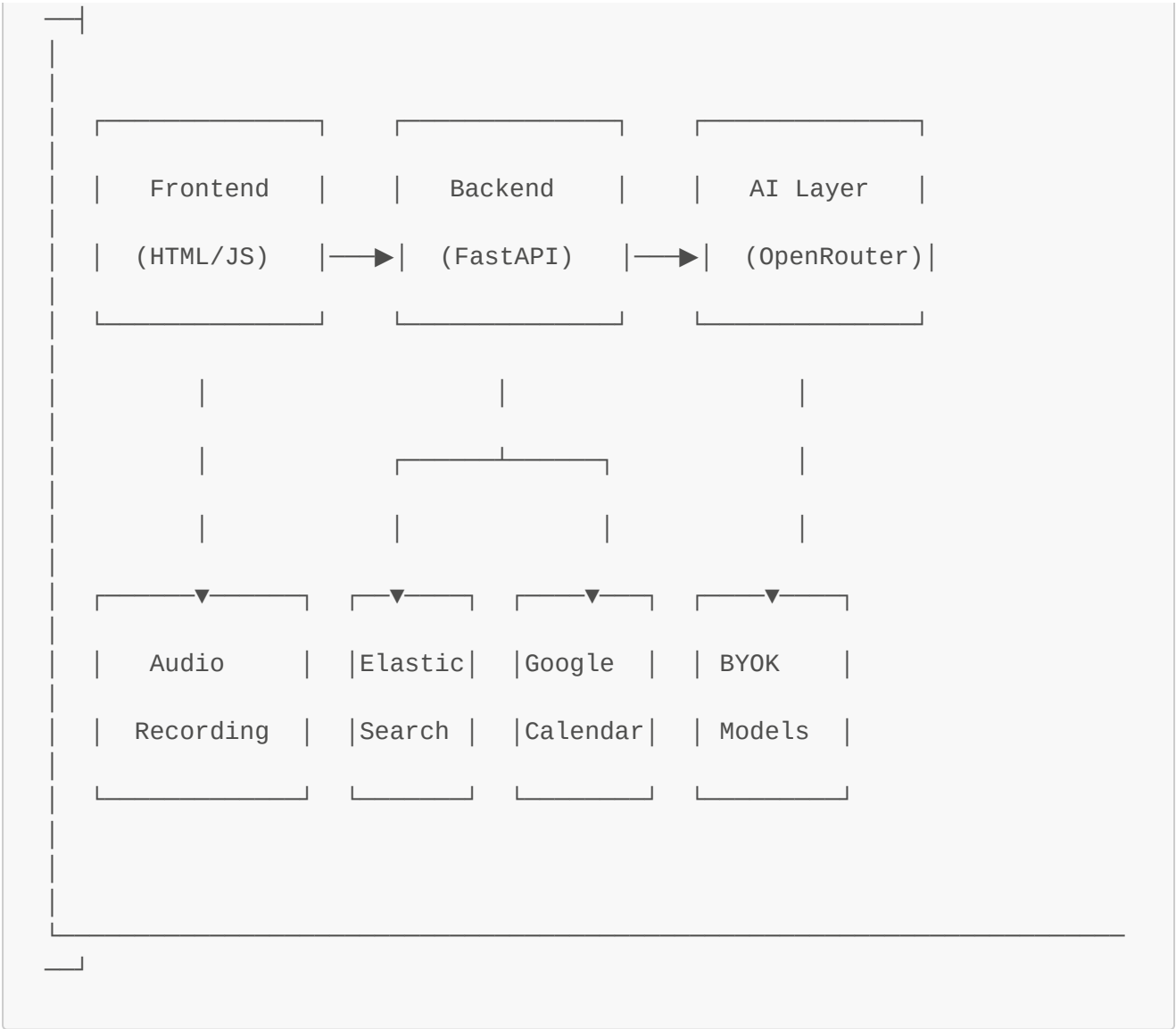
Competitor Weaknesses to Exploit

- Paras
- Fireflies/Otter:** Require pre-scheduled calls, no Hindi
 - Gemini:** Locked to Google ecosystem, no standalone mobile
 - Fathom:** No calendar/task integrations
 - Manus:** No task extraction, basic speaker ID
 - All:** No BYOK privacy option

☐ Solution Architecture

High-Level Architecture





Component Breakdown

1. Frontend Layer (PWA)

- **Technology:** HTML5 + CSS3 + Vanilla JS (mobile-first PWA)
- **Features:**
 - Audio recording via Web Audio API
 - Google OAuth login
 - Editable transcription/tasks/calendar UI
 - Offline capability (Service Worker)

2. Backend Layer (FastAPI)

- **Technology:** Python 3.11 + FastAPI
- **Features:**
 - Audio upload & processing
 - AI orchestration
 - User/team management
 - History & search

3. AI Layer (Pluggable)

- **Default:** OpenRouter (Claude 3.5 Sonnet)
- **BYOK Options:** OpenAI, Gemini, Groq
- **Features:**
 - Transcription (Whisper API or Deepgram)
 - Multi-language translation
 - Task extraction with deadlines
 - Calendar event generation
 - MoM composition

4. Data Layer (Elasticsearch)

- **Why Elasticsearch:**
 - Full-text search on transcriptions
 - Fast aggregations for analytics
 - Schema flexibility for meetings
 - Existing infrastructure on lehana.in

Data Flow

User Taps Record



Audio Capture
(webm/opus)

(Browser Web Audio API)



Upload to
Backend

POST /api/v1/meetings/upload



AI PROCESSING PIPELINE

1. Transcription (Whisper/Deepgram)

↳ Raw text with timestamps + speaker diarization

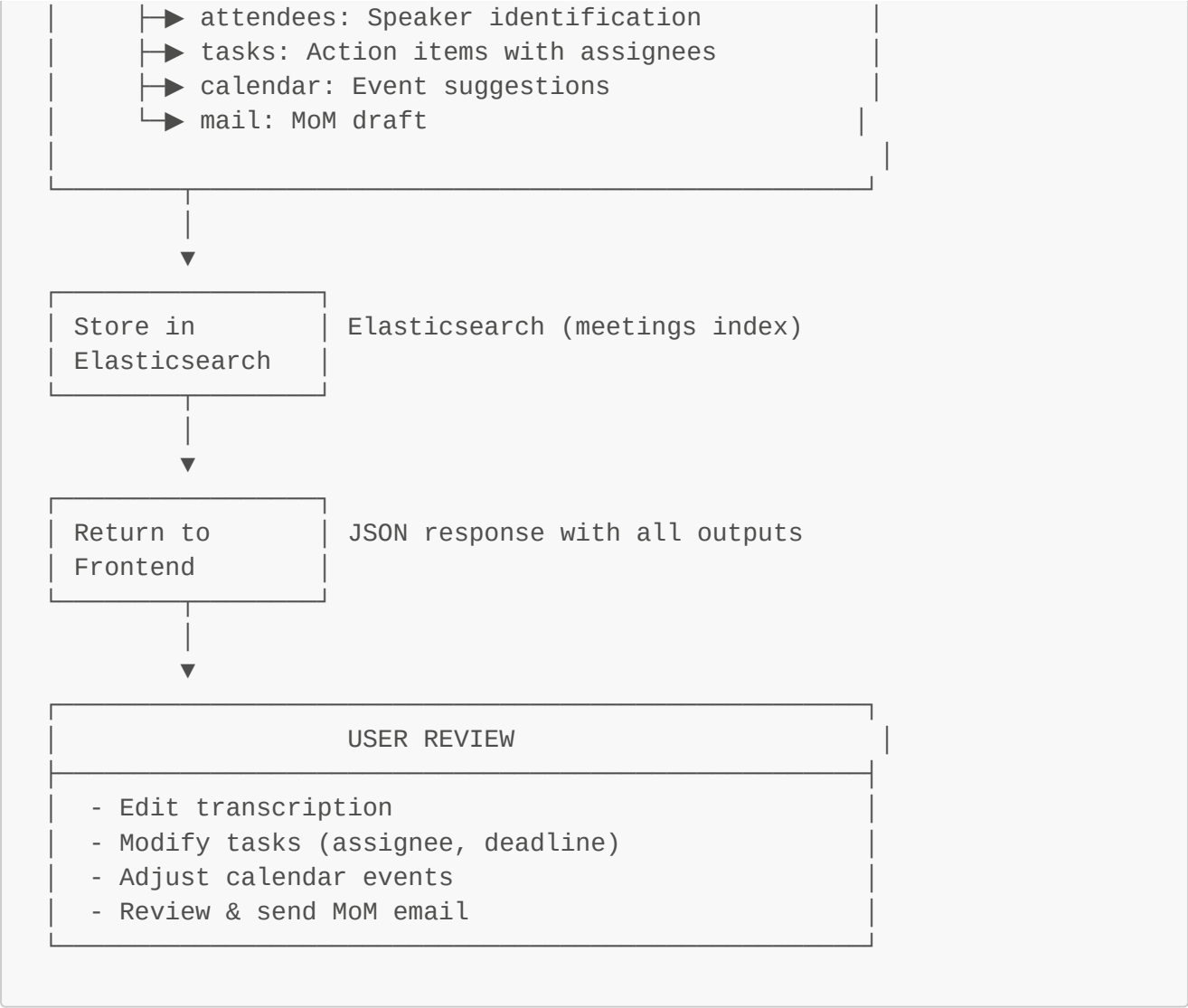
2. Translation (LLM)

↳ transcribe: English

↳ transcribe_hi: Hindi (Devanagari)

↳ transcribe_eh: Hinglish (English script)

3. Analysis (LLM)



□ Feature Specification

MVP Features (Phase 1)

Paras

Feature	Priority	Description
One-tap recording	P0	Start recording instantly from home screen
Audio upload	P0	Upload pre-recorded audio files
Manual transcript input	P0	Paste transcript if recording not available
AI transcription	P0	Convert audio to text (English, Hindi, Hinglish)
Speaker identification	P0	Label speakers (SPEAKER1, SPEAKER2, or names if detected)
Task extraction	P0	Extract action items with assignees
Smart deadlines	P0	AI-inferred deadlines based on context
Calendar events	P0	Generate calendar event suggestions
MoM email draft	P0	Editable email content for minutes

Feature	Priority	Description
Google OAuth	P0	Login with Google account
Attendee management	P1	Add/select meeting attendees
History	P1	View past meetings and search
BYOK models	P1	Use own API keys (Gemini, OpenAI, OpenRouter)

Future Features (Phase 2+)

Feature	Phase	Description
Offline recording	P2	Record without internet, sync later
WhatsApp sharing	P2	Send MoM via WhatsApp
Jira integration	P2	Create Jira tickets from tasks
Google Sheets sync	P2	Auto-fill task sheet
Team workspaces	P2	Shared team history
Voice commands	P3	"Assign this to Raj"
Sentiment analysis	P3	Team mood detection
Multi-meeting threads	P3	Link related meetings

AI Prompt Engineering

Core System Prompt

You are Meeting Master AI, an expert meeting assistant specializing in:

- Accurate transcription of multilingual meetings (English, Hindi, Hinglish)
- Professional speaker identification and attribution
- Intelligent task extraction with realistic deadlines
- Calendar event generation from meeting context
- Professional Minutes of Meeting (MoM) composition

Your outputs must be:

1. STRUCTURED: Always return valid JSON matching the specified schema
2. ACTIONABLE: Tasks must have clear owners and deadlines
3. CONTEXTUAL: Infer deadlines from project context when not explicit
4. PROFESSIONAL: MoM should be formal yet readable
5. MULTILINGUAL: Support Hindi, English, and code-mixed Hinglish

Transcription & Translation Prompt

TASK: Process the following meeting transcription and provide multilingual outputs.

INPUT TRANSCRIPTION:
{raw_transcription}

INSTRUCTIONS:

1. Clean up the transcription, fix obvious speech-to-text errors
2. Identify speakers (use names if mentioned, else SPEAKER_1, SPEAKER_2, etc.)
3. Translate to all three formats

OUTPUT JSON:

```
{
  "transcribe": "Full meeting in clean English. Speaker labels: [Paras]: ..., [Raj]: ...",
  "transcribe_hi": "पूरी मीटिंग हिंग्लिश में हिंदी लिपि में: [परास]: ..., [राज]: ...",
  "transcribe_eh": "Poori meeting Hinglish mein English script. Speaker labels: [Paras]: ..., [Raj]: ..."
}
```

RULES:

- Preserve all spoken content, don't summarize
- Keep speaker attributions consistent across all versions
- For Hindi translation, use natural Hindi (not transliteration)
- For Hinglish, use the code-mixed style common in Indian offices

Attendee Identification Prompt

TASK: Identify meeting attendees from the transcription.

INPUT:

Transcription: {transcription}
Known team members: {team_members_json}

INSTRUCTIONS:

1. Match speakers to known team members using name mentions, voice references
2. For unknown speakers, extract any identifying info (name, role, company)
3. Provide confidence hints for each identification

OUTPUT JSON:

```
{
  "attendees": [
    {
      "speaker_id": "SPEAKER_1",
      "name": "Paras",

```



```

        "email": "paras.lehana@indiamart.com",
        "identified": true,
        "hint": "Mentioned by name in transcription, matched to team
member"
    },
    {
        "speaker_id": "SPEAKER_2",
        "name": "Unknown - possibly client",
        "email": null,
        "identified": false,
        "hint": "Referred to as 'the vendor' by others, likely external"
    }
]
}

```

RULES:

- Use exact match with team members when name is mentioned
- Extract email if mentioned in meeting
- Provide helpful hints for manual correction

Task Extraction Prompt

TASK: Extract action items and tasks from the meeting.

INPUT:

Transcription: {transcription}

Meeting date: {meeting_date}

Known project context: {project_context}

Previous tasks from this project: {previous_tasks}

INSTRUCTIONS:

1. Identify ALL action items, commitments, and follow-ups
2. Assign to specific person (or "ALL" for team tasks)
3. INFER DEADLINES using these rules:
 - Explicit deadline: "by Friday" → use that date
 - Implicit urgency: "ASAP", "urgent" → within 2 days
 - Sprint context: If sprint end mentioned → sprint end date
 - Default: Task complexity estimate (simple: 3 days, medium: 1 week, complex: 2 weeks)
4. Categorize: BUG, FEATURE, DOCUMENTATION, MEETING, OTHER

OUTPUT JSON:

```

{
  "tasks": [
    {
      "id": "TASK_001",
      "title": "Fix login bug on mobile app",
      "description": "Users unable to login with Google OAuth on iOS",
      "assignee": "Raj",
      "deadline": "2026-01-31",

```

```

        "deadline_source": "explicit - mentioned 'by end of month'",
        "priority": "HIGH",
        "category": "BUG",
        "dependencies": [],
        "context": "Discussed in first 5 minutes of meeting"
    },
    {
        "id": "TASK_002",
        "title": "Prepare demo for client presentation",
        "description": "Create working demo of new feature for client
meeting",
        "assignee": "Paras",
        "deadline": "2026-02-03",
        "deadline_source": "inferred - client meeting mentioned as 'next
week Tuesday'",
        "priority": "MEDIUM",
        "category": "FEATURE",
        "dependencies": ["TASK_001"],
        "context": "Depends on login fix being complete"
    }
]
}

```

RULES:

- Every task MUST have a deadline (infer if not explicit)
- Use realistic estimates based on task complexity
- Link dependencies when tasks are mentioned together
- Include context for each task

Calendar Event Generation Prompt

TASK: Generate calendar events from meeting content.

INPUT:

Tasks: {tasks_json}
 Meetings mentioned: {meeting_mentions}
 Meeting date: {meeting_date}
 Attendees: {attendees_json}

INSTRUCTIONS:

1. Create calendar events for:
 - Explicitly mentioned future meetings
 - Task deadlines (as reminders)
 - Follow-up meetings implied by context
2. Suggest appropriate duration and time
3. Include relevant attendees

OUTPUT JSON:

```

{
  "calendar_events": [

```

```

{
  "id": "CAL_001",
  "title": "Client Demo Presentation",
  "description": "Demo new feature to client - ensure login fix is
deployed",
  "start_datetime": "2026-02-03T14:00:00",
  "end_datetime": "2026-02-03T15:00:00",
  "attendees": ["paras.lehana@indiamart.com", "raj@company.com"],
  "source": "explicit - mentioned in meeting",
  "type": "MEETING"
},
{
  "id": "CAL_002",
  "title": "DEADLINE: Login Bug Fix",
  "description": "Reminder - Raj to complete mobile login fix",
  "start_datetime": "2026-01-31T09:00:00",
  "end_datetime": "2026-01-31T09:30:00",
  "attendees": ["raj@company.com"],
  "source": "task deadline",
  "type": "REMINDER"
}
]
}

```

RULES:

- Explicit meetings get priority
- Task deadlines become morning reminders
- Default meeting duration: 30 min (standup), 60 min (discussion)
- Include context in description

MoM Email Prompt

TASK: Compose a professional Minutes of Meeting email.

INPUT:

Meeting title: {title}
 Date: {date}
 Attendees: {attendees}
 Transcription summary: {summary}
 Tasks: {tasks}
 Calendar events: {calendar_events}

OUTPUT JSON:

```

{
  "mail": {
    "subject": "MoM: Sprint Planning - January 29, 2026",
    "to": ["team@company.com"],
    "cc": [],
    "body": "Hi Team,\n\nPlease find the minutes of our meeting held on
January 29, 2026.\n\n## Attendees\n- Paras Lehana\n- Raj Kumar\n\n##

```

```
Discussion Summary\n[Summary of key points discussed]\n\n## Action Items\n| Task | Owner | Deadline |\n|-----|-----|-----|\n| Fix login bug | Raj | Jan 31 |\n| Prepare demo | Paras | Feb 3 |\n\n## Upcoming Events\n- Client Demo: Feb 3, 2:00 PM\n\nPlease review and let me know if anything needs correction.\n\nBest regards,\nMeeting Master AI"
```

RULES:

- Professional but friendly tone
- Markdown formatting for readability
- Table format for tasks
- Include all relevant details
- End with call to action

Combined Processing Prompt (Full Pipeline)

You are Meeting Master AI. Process this meeting recording and extract all information.

MEETING CONTEXT:

- Recording date: {date}
- Duration: {duration}
- Uploader: {uploader}
- Known team: {team_members}
- Project context: {project_context}

RAW TRANSCRIPTION:

{raw_transcription}

PROCESS AND RETURN A SINGLE JSON WITH ALL OUTPUTS:

```
{
  "transcribe": "...",
  "transcribe_hi": "...",
  "transcribe_eh": "...",
  "attendees": [...],
  "tasks": [...],
  "calendar_events": [...],
  "mail": {...},
  "summary": "2-3 sentence meeting summary",
  "tags": ["sprint-planning", "bug-discussion"],
  "sentiment": "positive|neutral|negative",
  "confidence": 0.85
}
```

Follow all the rules specified in individual prompt sections.

□ Database Schema (Elasticsearch)

Index: `meeting-master-users`

```
{
  "mappings": {
    "properties": {
      "user_id": { "type": "keyword" },
      "email": { "type": "keyword" },
      "name": { "type": "text" },
      "google_id": { "type": "keyword" },
      "picture_url": { "type": "keyword" },
      "created_at": { "type": "date" },
      "last_login": { "type": "date" },
      "settings": {
        "type": "object",
        "properties": {
          "default_language": { "type": "keyword" },
          "model_provider": { "type": "keyword" },
          "api_keys": {
            "type": "object",
            "properties": {
              "openai": { "type": "keyword" },
              "gemini": { "type": "keyword" },
              "openrouter": { "type": "keyword" }
            }
          }
        }
      }
    },
    "team_members": {
      "type": "nested",
      "properties": {
        "name": { "type": "text" },
        "email": { "type": "keyword" },
        "added_at": { "type": "date" }
      }
    }
  }
}
```

Paras

Index: `meeting-master-meetings`

```
{
  "mappings": {
    "properties": {
      "meeting_id": { "type": "keyword" },
      "user_id": { "type": "keyword" },
      "title": { "type": "text" },

```

```
"date": { "type": "date" },
"duration_seconds": { "type": "integer" },
"audio_url": { "type": "keyword" },
"status": { "type": "keyword" },

"transcribe": { "type": "text", "analyzer": "english" },
"transcribe_hi": { "type": "text", "analyzer": "hindi" },
"transcribe_eh": { "type": "text", "analyzer": "english" },

"attendees": {
  "type": "nested",
  "properties": {
    "speaker_id": { "type": "keyword" },
    "name": { "type": "text" },
    "email": { "type": "keyword" },
    "identified": { "type": "boolean" },
    "hint": { "type": "text" }
  }
},

"tasks": {
  "type": "nested",
  "properties": {
    "id": { "type": "keyword" },
    "title": { "type": "text" },
    "description": { "type": "text" },
    "assignee": { "type": "keyword" },
    "deadline": { "type": "date" },
    "priority": { "type": "keyword" },
    "category": { "type": "keyword" },
    "status": { "type": "keyword" }
  }
},

"calendar_events": {
  "type": "nested",
  "properties": {
    "id": { "type": "keyword" },
    "title": { "type": "text" },
    "start_datetime": { "type": "date" },
    "end_datetime": { "type": "date" },
    "attendees": { "type": "keyword" },
    "google_event_id": { "type": "keyword" }
  }
},

"mail": {
  "type": "object",
  "properties": {
    "subject": { "type": "text" },
    "body": { "type": "text" },
    "sent": { "type": "boolean" },
    "sent_at": { "type": "date" }
```

```

    }
  },

  "summary": { "type": "text" },
  "tags": { "type": "keyword" },
  "sentiment": { "type": "keyword" },
  "created_at": { "type": "date" },
  "updated_at": { "type": "date" }
}
}
}

```

□ API Design

Authentication

```

POST /api/v1/auth/google
- Input: Google OAuth token
- Output: JWT access token + user info

GET /api/v1/auth/me
- Header: Authorization: Bearer {token}
- Output: Current user profile

```

Meetings

```

POST /api/v1/meetings/upload
- Input: multipart/form-data (audio file) OR JSON (transcript text)
- Output: meeting_id, processing status

GET /api/v1/meetings/{meeting_id}
- Output: Full meeting data

GET /api/v1/meetings
- Query: ?page=1&limit=10&search=keyword
- Output: Paginated meeting list

PUT /api/v1/meetings/{meeting_id}
- Input: Updated meeting data (edits to tasks, calendar, etc.)
- Output: Updated meeting

DELETE /api/v1/meetings/{meeting_id}
- Output: Success/failure

```

Processing

```
POST /api/v1/meetings/{meeting_id}/process
- Input: { "model_provider": "openrouter", "options": {...} }
- Output: Processing job ID

GET /api/v1/meetings/{meeting_id}/status
- Output: Processing status and progress
```

Team Management

```
GET /api/v1/team
- Output: List of team members

POST /api/v1/team
- Input: { "name": "Raj", "email": "raj@company.com" }
- Output: Added team member

DELETE /api/v1/team/{member_email}
- Output: Success/failure
```

Settings

```
GET /api/v1/settings
- Output: User settings

PUT /api/v1/settings
- Input: Updated settings (API keys, preferences)
- Output: Updated settings
```

Calendar Integration

```
POST /api/v1/calendar/events
- Input: { "meeting_id": "...", "event_ids": ["CAL_001", "CAL_002"] }
- Output: Created Google Calendar events

GET /api/v1/calendar/authorize
- Output: Google Calendar OAuth URL
```

□ Frontend Design

Screen Flow

HOME SCREEN

[User Avatar] Paras Lehana

[Settings ⚙]

🎙 TAP TO RECORD

[Large circular button]

Or paste/type your transcript:

[Textarea for manual input]

[Process Transcript →]

Recent Meetings

📅 Sprint Planning - Jan 28

[View]

📅 Client Call - Jan 27

[View]

📅 Standup - Jan 27

[View]

Paras

Recording Screen

RECORDING IN PROGRESS

🕒 03:45

[Animated waveform visualization]

Meeting Title: [Sprint Planning_____]

Attendees: [+ Add]

Paras

Raj

+Add

[⏸ Pause]

[⏹ Stop & Process]

Results Screen

MEETING RESULTS

Sprint Planning - January 29, 2026

Duration: 15 minutes | Attendees: 3

[📄 Transcript] [📋 Tasks] [📅 Calendar] [✉ Email]

TAB: TRANSCRIPT

Language: [English ▼] [Hindi] [Hinglish]

[Paras]: Okay team, let's start with the sprint...

[Raj]: Sure, I've been working on the login fix...

[Priya]: The client demo is scheduled for Tuesday...

[Edit Transcript]

TAB: TASKS

☐ Fix login bug

Assignee: [Raj ▼] Deadline: [Jan 31 📅]

☐ Prepare client demo

Assignee: [Paras ▼] Deadline: [Feb 3 📅]

[+ Add Task]

TAB: CALENDAR

☒ Client Demo - Feb 3, 2:00 PM

Attendees: Paras, Raj, Priya

☒ Deadline: Login Bug Fix - Jan 31

Reminder for: Raj

[+] Add to Google Calendar

TAB: EMAIL

To: team@company.com

Subject: MoM: Sprint Planning - Jan 29, 2026

Hi Team,

Please find the minutes of our meeting...

[Edit Email] [+] Send Email [+] Share WhatsApp

Settings Screen

SETTINGS

Account

[+] Paras Lehana

[+] paras.lehana@gmail.com

[Logout]

AI Model

○ Use Meeting Master AI (Default)

○ Use my own API keys

[Collapsed: API Key Settings]

OpenAI Key: [sk-..._____] [Test]

Gemini Key: [AI..._____] [Test]

OpenRouter: [sk-or..._____] [Test]

Team Members

Raj Kumar - raj@company.com [X]

Priya Singh - priya@company.com [X]

[+ Add Team Member]

Preferences

Default Language: [English ▼]
Auto-process recordings: [✓]
Send MoM copies to myself: [✓]

□ Technology Stack

Backend

- **Framework:** FastAPI 0.109+
- **Python:** 3.11+
- **Audio Processing:** pydub, ffmpeg
- **AI/LLM:** OpenRouter API, OpenAI API, Google Generative AI
- **Database:** Elasticsearch 8.x
- **Auth:** Google OAuth 2.0, python-jose (JWT)
- **Task Queue:** Background tasks (FastAPI) / Celery (future)

Frontend

- **Core:** HTML5, CSS3, Vanilla JavaScript
- **Audio:** Web Audio API, MediaRecorder API
- **PWA:** Service Worker, Web App Manifest
- **Styling:** Custom CSS with CSS Variables (dark theme)
- **Charts:** Chart.js (analytics)

Infrastructure

Paras

- **Container:** Docker + Docker Compose
- **Reverse Proxy:** Traefik v3
- **DNS:** Cloudflare (lehana.in + aidhunik.com)
- **SSL:** Let's Encrypt wildcard
- **Storage:** Local volume mounts

Third-Party Services

- **Transcription:** Deepgram API / Whisper API
- **LLM:** OpenRouter (Claude 3.5), OpenAI (GPT-4), Google (Gemini)
- **Calendar:** Google Calendar API
- **Email:** SMTP / SendGrid (future)

□ Deployment Plan

Phase 1: MVP (This Session)

1. ☐ Create DESIGN.md
2. ☐ Build backend API
3. ☐ Build frontend UI
4. ☐ Docker configuration
5. ☐ Deploy to meeting.lehana.in
6. ☐ Test end-to-end

Phase 2: Polish (Next Session)

1. Google Calendar integration
2. Email sending capability
3. Team management
4. History search
5. BYOK model support

Phase 3: Scale (Future)

1. Offline mode
2. WhatsApp sharing
3. Jira/Sheets integration
4. Mobile app (React Native)
5. Team workspaces

☐ Success Metrics

Technical KPIs

- Transcription accuracy: > 90%
- Task extraction precision: > 85%
- API response time: < 5s (processing) / < 100ms (queries)
- Uptime: 99.9%

Business KPIs

- User signups: 100 in first month
- Active users: 50% weekly retention
- Meetings processed: 1000/month
- Conversion to premium: 10%

☐ Next Steps

IMMEDIATE: Start building the backend API with all endpoints and AI integration.

Files to Create:

1. `/root/ideas/meeting-master/backend/api.py` - Main FastAPI application
2. `/root/ideas/meeting-master/backend/models.py` - Pydantic models
3. `/root/ideas/meeting-master/backend/services/ai.py` - AI processing
4. `/root/ideas/meeting-master/backend/services/elastic.py` - Database
5. `/root/ideas/meeting-master/frontend/index.html` - Main UI
6. `/root/ideas/meeting-master/docker/docker-compose.yml` - Deployment

LET'S BUILD! 🚀